บทที่ 8: ความท้าทายของระบบกระจาย (The Trouble with Distributed Systems)

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 15–18 นาที ฟังเพลิน ๆ ก่อนนอนได้ครับ)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่แปด ว่าด้วยความท้าทายของระบบกระจาย

ตั้งแต่บทที่ห้าเป็นต้นมา เราพูดถึงระบบกระจายมาตลอด เช่น การทำสำเนาข้อมูลและการแบ่ง partition แต่เรามักแกล้งทำเป็นมองข้ามความจริงข้อหนึ่งไป คือ โลกของระบบกระจายนั้นเต็มไปด้วยความไม่แน่นอน

ถ้าโปรแกรมรันอยู่บนเครื่องเดียว ทุกอย่างค่อนข้างตรงไปตรงมา หน่วยความจำมีค่าเดียว ซีพียูก็รู้ลำดับเวลาชัดเจน แต่พอแยกออกไปอยู่หลายเครื่อง ผ่านเครือข่ายที่ไม่รู้ว่าเมื่อไรจะส่งข้อมูลถึง ความวุ่นวายสารพัดรูปแบบก็เริ่มต้นขึ้น

บทนี้จะพาไปดูความจริงอันโหดร้ายของระบบกระจาย ว่าทำไมเครือข่ายถึงไว้ใจไม่ได้ ทำไมนาฬิกาของแต่ละเครื่องถึงไม่ตรงกัน และเราจะเอาตัวรอดจากความป่วนเหล่านี้ได้อย่างไรครับ

ตอนที่ 2: ความผิดพลาดของเครือข่าย

ในระบบเครื่องเดียว คอมพิวเตอร์คุยกันผ่านบัสภายใน ซึ่งแทบไม่เคยพลาด แต่ในระบบกระจาย เครื่องคอมพิวเตอร์คุยกันผ่านเครือข่าย ซึ่งเป็นแบบ asynchronous network

ปัญหาคลาสสิกของเครือข่ายคือ ถ้าเราส่งข้อความไปแล้วอีกฝ่ายไม่ตอบกลับมา เราไม่มีทางรู้เลยว่าเกิดอะไรขึ้น

- ข้อความของเรายังเดินทางไปไม่ถึง (คำขอหาย)
- อีกฝ่ายได้รับแล้ว กำลังประมวลผลอยู่แต่ช้ามาก
- อีกฝ่ายประมวลผลเสร็จแล้ว แต่ตอนส่งคำตอบกลับมา คำตอบดันหายไปบนเครือข่าย

ความคลุมเครือนี้ทำให้ระบบกระจายปวดหัวที่สุด เพราะเราตอบไม่ได้ว่า "อีกฝั่งพังหรือยัง"

วิธีแก้ปัญหามาตรฐานที่ใช้กันคือ Timeouts หรือการตั้งเวลารอ ถ้าหมดเวลาแล้วยังไม่ตอบ ก็ถือว่ามีปัญหา แล้วค่อยลองส่งใหม่ (Retry)

แต่การลองส่งใหม่ก็สร้างปัญหาอื่นตามมา เช่น สมมติเราส่งคำสั่งตัดเงินผ่านเครือข่าย แล้วหมดเวลาก่อนได้คำตอบ เราลองส่งซ้ำรอบสอง ปรากฏว่ารอบแรกจริง ๆ แล้วสำเร็จไปแล้ว ลูกค้าก็โดนตัดเงินสองรอบทันที ปัญหานี้แก้ไขด้วยเทคนิคที่เรียกว่า Idempotence คือออกแบบให้คำสั่งไม่ว่าจะส่งซ้ำกี่ครั้ง ผลลัพธ์ก็เหมือนเดิม เช่น แทนที่จะบอกว่า "ให้บวกเงินเพิ่ม 10 บาท" ให้เปลี่ยนเป็น "ให้ตั้งยอดเงินเป็น 110 บาท" แบบนี้ส่งซ้ำกี่รอบก็ปลอดภัย

ความผันผวนของเครือข่าย

ไม่ใช่แค่หาย แต่เครือข่ายยังชอบแกล้งทำเป็นช้าด้วย บางครั้งข้อมูลไปกองรวมกันจนเกิดความคับคั่ง (Congestion) ทำให้เวลาเดินทางกระโดดจากไม่กี่มิลลิวินาทีกลายเป็นหลายวินาที

ระบบส่วนใหญ่จึงมักใช้การทดสอบแบบสุ่ม (Randomized testing) หรือสุ่มจับเวลา เพื่อดูว่าเครื่องไหนล่มจริง หรือแค่เครือข่ายหน่วงชั่วคราว

ตอนที่ 3: นาฬิกาบอกเวลาที่ไม่เคยตรงกัน

เวลาเราดูบันทึกข้อผิดพลาด (Log) เราคาดหวังว่าเหตุการณ์ที่เกิดขึ้นก่อน จะต้องมีเวลา (Timestamp) น้อยกว่าเหตุการณ์ที่เกิดทีหลัง แต่ในระบบกระจาย เรื่องนี้เป็นไปไม่ได้เลยถ้าใช้นาฬิกาปกติ

เครื่องคอมพิวเตอร์ทุกเครื่องมีนาฬิกาในตัวเรียกว่า Hardware Clock (มักเป็น Crystal oscillator) ซึ่งมีความคลาดเคลื่อนเสมอ เดินเร็วหรือช้าไม่เท่ากัน แม้เราจะพยายามตั้งเวลาให้ตรงกันผ่านอินเทอร์เน็ตด้วยโปรโตคอลอย่าง NTP (Network Time Protocol) แต่นาฬิกาก็ยังกระโดด เดินหน้า หรือถอยหลังเองได้ตามอุณหภูมิและความเสื่อมของอุปกรณ์

อันตรายของการใช้นาฬิกาจริงกับระบบกระจาย

สมมติเรามีระบบที่มีสำเนาสองเครื่อง (A และ B) บันทึกการเขียนข้อมูลลงไป

- เครื่อง A ได้รับการเขียนข้อมูลตอน 12:00:01 ตามนาฬิกาของมัน
- เครื่อง B ได้รับการเขียนข้อมูลตอน 12:00:02 ตามนาฬิกาของมัน (แต่นาฬิกาเครื่อง B เดินเร็วกว่าความจริงไป 2 วินาที)

ถ้าเราเอาค่าเวลามาเทียบกันเพื่อดูว่าอันไหนเกิดก่อน เราจะสรุปว่าเครื่อง B เกิดทีหลัง ทั้งที่ในความเป็นจริง เครื่อง B เกิดก่อนหน้าเครื่อง A ข้อมูลจะตีกันเละเทะทันที

แนวคิดแบบ Last Write Wins (LWW) ที่ใช้ในฐานข้อมูลหลายตัว (เช่น Cassandra) อาศัยเวลาจากนาฬิกาเครื่องนี่แหละครับเป็นตัวตัดสินว่าค่าไหนชนะ ซึ่งผู้เขียนเตือนตัวโต ๆ เลยว่า อันตรายมาก เพราะถ้าเครื่องไหนนาฬิกาเพี้ยน ข้อมูลใหม่กว่าอาจถูกลบทิ้งอย่างไร้เยื่อใยเพราะแพ้เวลาของเครื่องอื่น

การซิงก์เวลาแบบปลอดภัย

ถ้าจำเป็นต้องใช้นาฬิกาจริงจริง ๆ ระบบต้องมีขอบเขตความคลาดเคลื่อน เช่น Google Spanner ใช้ฮาร์ดแวร์พิเศษคือ นายูนิเซฟ เอ้ยไม่ใช่ ใช้ GPS receiver และ Atomic clock ติดตั้งไว้ที่ศูนย์ข้อมูล เพื่อจำกัดความคลาดเคลื่อนให้น้อยที่สุด แล้วระบบจะนำช่วงความคลาดเคลื่อนนั้นมาคำนวณรอ (Wait out) เพื่อให้มั่นใจจริง ๆ ว่าเหตุการณ์ A เกิดก่อน B แน่นอน แต่ระบบแบบนี้มีต้นทุนสูงมาก คนทั่วไปทำตามได้ยาก

ตอนที่ 4: ความจริง ความหลอกลวง และความเห็นพ้อง

ในระบบกระจาย เราไม่มีทางรู้ความจริงร้อยเปอร์เซ็นต์ ทุกอย่างขับเคลื่อนด้วย "มุมมอง" ของแต่ละเครื่อง ถ้าเครื่องส่วนใหญ่บอกว่าเห็นแบบนั้น เราก็ต้องถือว่านั่นคือความจริง

ความจริงของคนส่วนใหญ่ (Quorum) เหมือนที่เราคุยกันในบทการทำสำเนา ถ้าเครื่องส่วนใหญ่ (เช่น 3 ใน 5 เครื่อง) ลงความเห็นตรงกัน เราจะถือว่าข้อมูลนั้นถูกต้อง

ปัญหาการแบ่งแยกสมอง (Split Brain) ถ้าเครือข่ายขาดตอนกลางคัน ทำให้ระบบถูกตัดแบ่งเป็นสองฝั่ง ฝั่งซ้ายมี 2 เครื่อง ฝั่งขวา 3 เครื่อง ต่างฝ่ายต่างคิดว่าตัวเองคือผู้ใหญ่บ้าน และต่างฝ่ายต่างรับเขียนข้อมูลจากผู้ใช้ทั้งคู่ แบบนี้ข้อมูลจะพังเสียหายยับเยิน

เพื่อป้องกันปัญหานี้ ระบบจึงต้องมีกลไกการโหวตเลือกข้างที่เด็ดขาด เรียกว่า Consensus (ความเห็นพ้องต้องกัน) ซึ่งต้องอาศัยเสียงข้างมากที่แท้จริง (Strict Majority มากกว่าครึ่งหนึ่งของระบบทั้งหมดเสมอ) ถ้าฝั่งไหนมีเครื่องไม่ถึงครึ่งจะไม่ยอมตัดสินใจอะไรเด็ดขาดเพื่อความปลอดภัย

ตอนที่ 5: กระบวนการเห็นพ้อง (Consensus)

Consensus คือปัญหาคลาสสิกและยิ่งใหญ่ที่สุดในวิทยาการคอมพิวเตอร์สายระบบกระจาย กล่าวสั้น ๆ คือ ทำอย่างไรให้เครื่องหลาย ๆ เครื่องในระบบที่ไว้ใจไม่ได้ ตกลงเรื่องเดียวกันให้ได้

อัลกอริทึมที่ใช้แก้ปัญหานี้ชื่อดังมีอยู่ไม่กี่ตัว เช่น Paxos, Raft และ Zab (ZooKeeper Atomic Broadcast)

หลักการคร่าว ๆ ของอัลกอริทึมเหล่านี้คือ

1. รับประกันว่าถ้าเครื่องส่วนใหญ่ตกลงเลือกค่าหนึ่งแล้ว ทุกเครื่องจะต้องจำค่านั้นเหมือนกันหมด
2. จะไม่มีการตัดสินใจสองแบบที่ขัดแย้งกันเองในระบบ
3. รับประกันความก้าวหน้า คือตราบใดที่เครื่องส่วนใหญ่ยังไม่ตาย ระบบจะต้องหาข้อสรุปและเดินหน้าต่อไปได้

ระบบจัดการคลังข้อมูลและการทำสำเนารุ่นใหม่ ๆ เกือบทั้งหมดพึ่งพาอัลกอริทึมกลุ่มนี้เบื้องหลัง เพื่อให้มั่นใจว่าผู้นำระบบจะไม่ถูกแต่งตั้งซ้ำซ้อน และข้อมูลทุกชุดถูกบันทึกลำดับอย่างถูกต้องตรงกัน

ตอนที่ 6: สรุปท้ายบท

มาทบทวนประเด็นสำคัญของบทนี้กันครับ

รวบยอดสั้น ๆ ก่อนนอน: หนึ่ง เครือข่ายในระบบกระจายไว้ใจไม่ได้ ข้อความอาจหาย ช้า หรือสลับลำดับ การตั้งเวลา (Timeout) และการออกแบบคำสั่งให้ทำซ้ำได้ปลอดภัย (Idempotence) จึงเป็นเรื่องจำเป็น

สอง นาฬิกาของแต่ละเครื่องไม่เคยตรงกัน การใช้เวลาจริงจากนาฬิกาเครื่องเพื่อมาตัดสินว่าอะไรเกิดก่อนเกิดหลังมีความเสี่ยงสูงมาก ข้อมูลอาจหายได้

สาม ความจริงในระบบกระจายเกิดจากการลงความเห็นของคนส่วนใหญ่ (Quorum และ Consensus) เพื่อป้องกันภาวะสมองแยก (Split Brain)

สี่ อัลกอริทึมอย่าง Raft และ Paxos คือหัวใจสำคัญที่ช่วยให้เครื่องหลาย ๆเครื่องตกลงเรื่องเดียวกันได้อย่างปลอดภัย

บทต่อไปคือบทที่เก้า ว่าด้วยเรื่องระบบประมวลผลแบบสตรีม (Stream Processing) ซึ่งจะพาเราเข้าสู่ภาคสุดท้ายของหนังสือเล่มนี้แล้วครับ พักผ่อนให้เต็มที่นะครับ ฝันดีครับ 😊